

Chapter 02

Struktur dan Logika Bahasa

Pemrograman

Pemrograman Terstruktur – Rosihan Ari Yuana, S.Si, M.Kom

Tujuan Pembelajaran

- Memahami perbedaan interpreter dan compiler
- Memahami struktur bahasa pemrograman
- Memahami logika berpikir bahasa pemrograman khususnya pada *flow control* (sekuensial, percabangan, dan perulangan)

Interpreter vs Compiler

Proses penterjemahan bhs program ke bhs mesin -> compiler dan interpreter

Interpreter	Compiler
Menterjemahkan setiap file kode program terpisah secara langsung	Beberapa file kode program digabung menjadi satu kemudian diterjemahkan menjadi sebuah file object
Proses penterjemahannya cepat	Proses penterjemahannya agak lama
Running programnya lebih lambat	Running programnya lebih cepat
Selama proses penterjemahan, memori yang digunakan lebih kecil	Selama proses penterjemahan, memori yang digunakan lebih lama
Mudah dalam proses debugging karena dilakukan per file kode program	Agak sulit dalam proses debugging

Interpreter vs Compiler

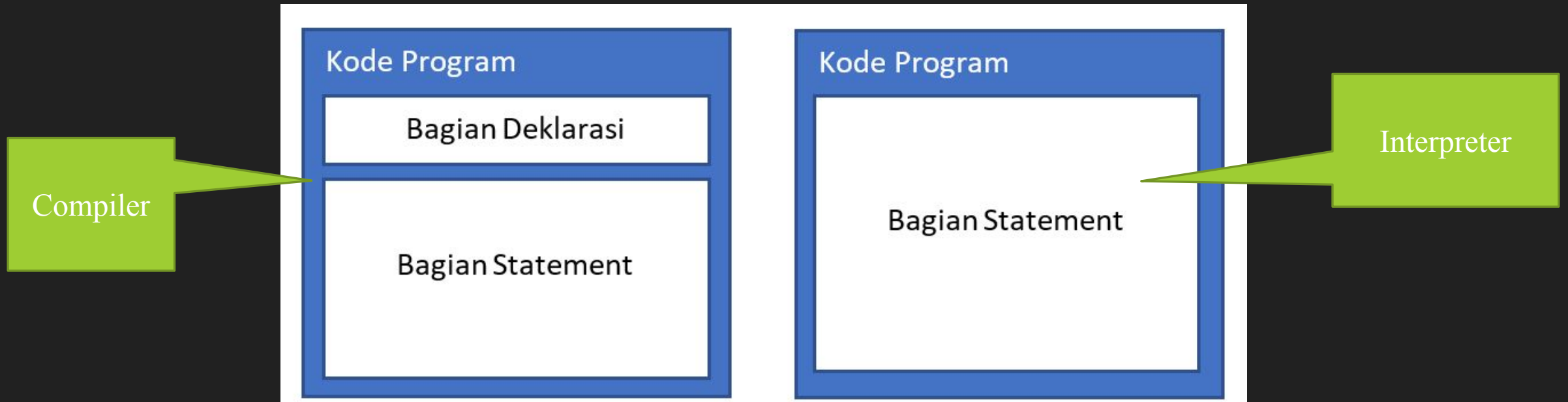
- Menggunakan interpreter: PHP, Python, Ruby, Perl
- Menggunakan compiler: Java, Pascal, C/C++, Delphi, VB

Struktur Bahasa Pemrograman

Kriteria kode program yang baik:

- ❑ Program harus sesuai dengan spesifikasi kebutuhan
- ❑ Fleksibel
- ❑ Tidak mengandung kesalahan
- ❑ Terdokumentasi secara baik
- ❑ Cepat dalam waktu penggunaannya
- ❑ Efisien dalam penggunaan memori komputer

Struktur Bahasa Pemrograman



Struktur bahasa pemrograman

Struktur Bahasa Pemrograman

Bag deklarasi

Bag statement

```
Language: Pascal All Gates Use Labels Text only Zoom: 100% Save Code

{ Code generated by GeneXproTools 4.0 on 13/07/2006 20:21:30 }
{ Training Samples: 64 }
{ Fitness Function: Sensitivity/Specificity }
{ Training Fitness: 1000.156 }
{ Training Accuracy: 100.00% }

program gepModel(input,output);
var
  d: array [0..5] of boolean;
  blTemp, result: boolean;
begin
  blTemp := false;

  blTemp := (d[5] or (not((d[1] and d[0]))));
  blTemp := blTemp and ((d[3] or d[0]) or (not(d[1])));
  blTemp := blTemp and ((d[1] or d[0]) or d[2]);
  blTemp := blTemp and (d[5] or (not(d[5])));
  blTemp := blTemp and ((d[4] or (not(d[0])) or d[1]);

  result := blTemp;
end.
```

Struktur program Pascal

```
Menu toggle_led.py - /home/pi/D...
File Edit Format Run Options Windows Help

import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BOARD)

outputpin = 7

GPIO.setup(outputpin,GPIO.OUT)

flipflop = True

while True:
    if flipflop:
        flipflop = False
        GPIO.output(outputpin,GPIO.HIGH)
        print("light on")
    else:
        flipflop = True
        GPIO.output(outputpin,GPIO.LOW)
        print ("light off")
    time.sleep(1)
```

Struktur program Python

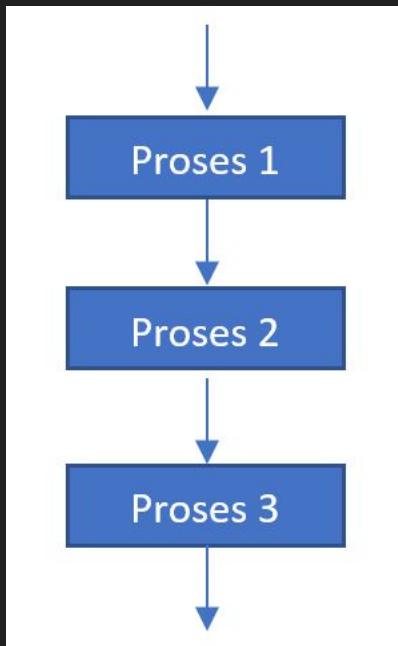
Logika Bahasa Pemrograman

- Logika dalam program mengadopsi logika berpikir manusia
- Alur proses dalam program -> sekuensial, percabangan, perulangan (looping)
- Alur proses yang salah -> algorithmic error

Logika Bahasa Pemrograman

Sekuensial (berurutan)

Semua bahasa program, instruksinya ditulis secara sekuensial (top - down)



Contoh: Langkah (algoritma) membuat mie rebus

1. Siapkan panci
2. Siapkan air sebanyak 2 gelas dan masukkan ke dalam panci
3. Rebus air dalam panci hingga mendidih
4. Masukkan mie bungkus ke dalam panci
5. Aduk-aduk selama 2-3 menit
6. Masukkan bumbu mie dan aduk-aduk mie sampai kira-kira 1 menit
7. Mie siap dihidangkan

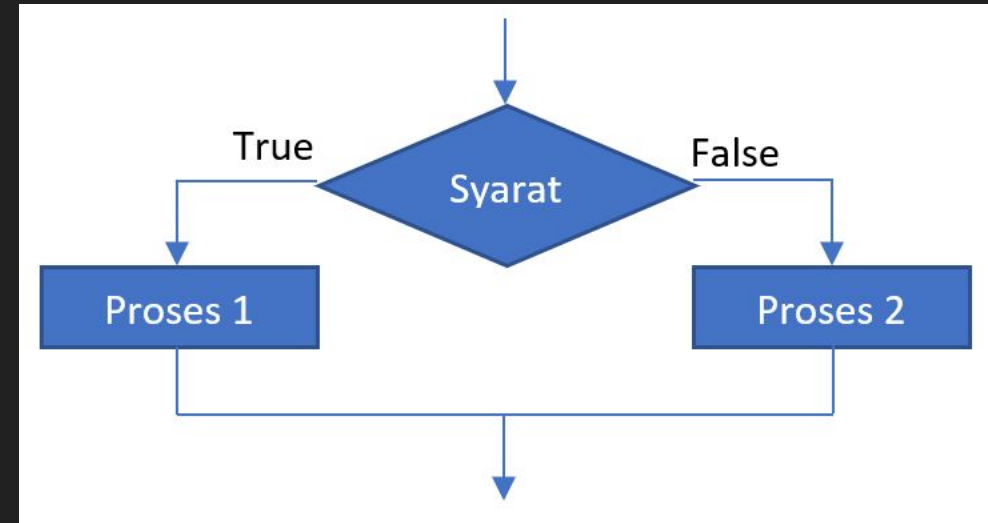
Logika Bahasa Pemrograman

Percabangan (Kondisional/Bersyarat)

Sebuah instruksi akan dijalankan apabila suatu syarat terpenuhi.

Contoh:

1. Siapkan panci
2. Siapkan air sebanyak 2 gelas dan masukkan ke dalam panci
3. Rebus air dalam panci
4. Jika air sudah mendidih, maka masukkan mie bungkus ke dalam panci
5. Jika belum, maka tunggu sampai mendidih
6. Aduk-aduk selama 2-3 menit
7. Masukkan bumbu mie dan aduk-aduk mie sampai kira-kira 1 menit
8. Mie siap dihidangkan



Logika Bahasa Pemrograman

Perulangan (Looping)

Instruksi yang dilakukan secara berulang-ulang

Contoh:

1. Siapkan cangkir atau gelas
2. Masukkan 1-2 sendok teh bubuk kopi
3. Tuangkan air panas ke dalam cangkir atau gelas
4. Masukkan gula ke dalam cangkir atau gelas
5. Aduk-aduk
6. Cicipi kopinya
7. Jika masih belum manis, maka ulangi langkah 4 sampai dengan 6
Jika sudah manis, maka lanjutkan langkah 8
8. Kopi manis siap dinikmati

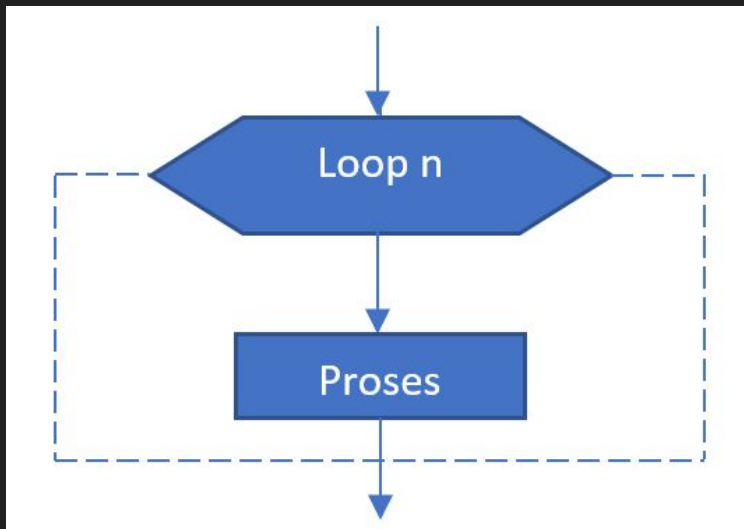
Logika Bahasa Pemrograman

Perulangan (Looping)

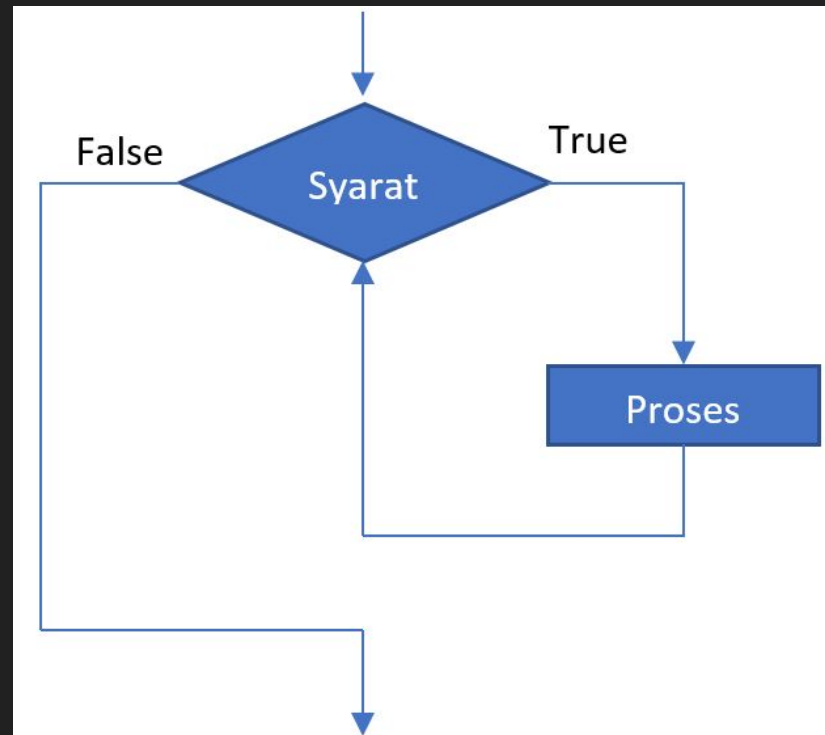
Beberapa bentuk perulangan:

1. Saya menuangkan segelas air ke dalam ember sebanyak 10 kali
2. Selama ember belum penuh, saya akan tetap menuangkan segelas air ke dalamnya
3. Saya terus menuangkan segelas air ke dalam ember hingga embernnya penuh

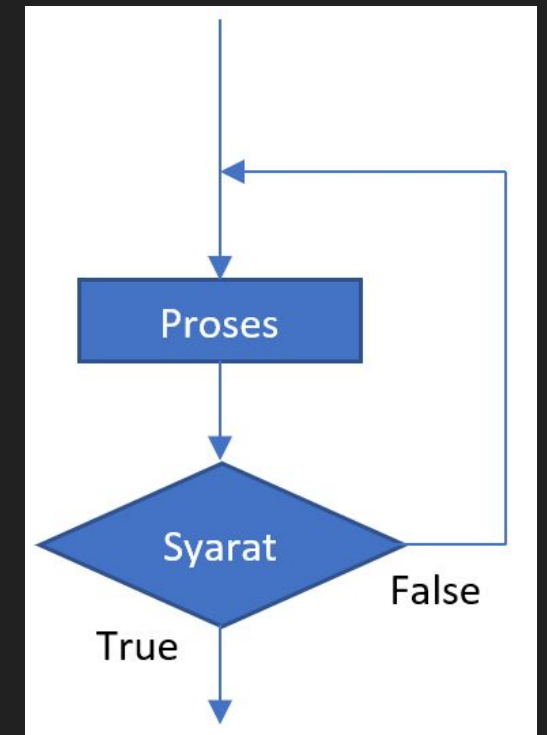
Logika Bahasa Pemrograman



Tipe 1



Tipe 2



Tipe 3

Logika Bahasa Pemrograman

```
for i in range(5):  
    print("Hello world")
```

(a)

```
i = 1  
while (i < 5):  
    print("Hello world")  
    i = i + 1
```

(b)

```
i = 1  
while (True):  
    print("Hello world")  
    i = i + 1  
    if (i > 4):  
        break
```

(c)

Contoh penggunaan bentuk perulangan di Python